Practical Task 3.2

(Distinction Task)

Submission deadline: Wednesday, April 8
Discussion deadline: Friday, April 24

General Instructions

This task extends Pass Task 3.1. You will need to implement three advanced algorithms: the Quick Sort, the Top-Down Merge Sort, and the Bottom-Up Merge Sort. You must program these algorithms in their respective classes, with each class implementing the enclosed ISorter interface. Because the subject of this task is based on Lecture 3, we strongly recommend you watch it before you start. The aspects of algorithms' implementations and their pseudocodes are given in Chapter 12 of the "Data structures and algorithms in Java" by Michael T. Goodrich, Irvine Roberto Tamassia, and Michael H. Goldwasser (2014), which is the SIT221 course book that you may access on-line from the reading list application in CloudDeakin by navigating to Content → Reading List.

1. You may extend your existing solution for Task 3.1 and replace the Tester.cs file by that attached to this task. The Main method of the new Tester class is to be compatible with your Vector<T> class. Again, it contains a series of tests that will verify if the functionality that you need to develop in this task works correctly. See Appendix A for an example of what the program will output when working right.

2. First, implement the Randomized Quick Sort algorithm in a new **RandomizedQuickSort** class. This recursive algorithm must perform sorting operations "*in-place*" and select the pivot element uniformly at random from the range of elements fed as an input to its current recursive call. The details of this algorithm can be found in Chapter 12.2 of the aforementioned SIT221 course book. Specifically, explore Section 12.2.2 that describes the *in-place* version of the Quick Sort and Code Fragment 12.6 that provides its pseudocode solution. To randomize the strategy for the pivot point selection, you will need to alter line 7 of Code Fragment 12.6 to introduce the logic described in Section 12.2.1. Keep in mind that generation of random numbers can be done via the standard *Random* class of the .NET Framework.

   Once your Quick Sort algorithm is ready, you should be able to sort integers using various comparison classes available in the Tester.cs file and succeed with tests A, B, and C. To enable these tests, do not forget to uncomment the respective code lines in the Main method of the Tester class.

3. Now, implement the Top-Down Merge Sort algorithm in a new **MergeSortTopDown** class. Here, you will need to follow the algorithm's description given in Section 12.1 of the SIT221 course book, and specifically Code Fragments 12.1 and 12.2 to implement its array-based version. Before you start, make sure you understand its recursive nature described in Section 12.1.1. When your algorithm works correctly, you should be able to activate and pass tests D, E, and F.

4. Finally, implement the bottom-up version of the Merge Sort algorithm in a new **MergeSortBottomUp** class. This algorithm is described in Section 12.1.5 of the SIT221 course book. Before you start, make sure you understand all the issues related to the use of temporary memory required to run merge operations. When your algorithm works correctly, you should be able to activate and pass tests G, H, and I.

5. This final step is crucial. You must test your sorting algorithms independently of the Vector<T> class. To do this, extend the existing set of tests by adding new cases with focus on sorting an array of data elements (e.g., integers or strings) by a direct call of the Sort methods of the **RandomizedQuickSort**, **MergeSortTopDown**, and **MergeSortBottomUp** classes. Be creative and think about the tests to be most effective in testing the algorithms for logical errors and runtime exceptions. Later, in a face-to-

face discussion with your tutor, you will need to clearly communicate your testing strategies and explain the purpose of proposed tests.

Submission Instructions and Marking Process

To get your task completed, you must finish the following steps strictly on time.

- Make sure your programs implement the required functionality. They must compile, have no runtime errors, and pass all test cases of the task. Programs causing compilation or runtime errors will not be accepted as a solution. You need to test your programs thoroughly before submission. Think about potential errors where your programs might fail.
- **Submit** the expected code files as a solution to the task via OnTrack submission system.
- Once your solution is accepted by your tutor, you will be invited to **continue its discussion and answer relevant theoretical questions through a face-to-face interview**. Specifically, you will need to meet with the tutor to demonstrate and discuss the solution in one of the dedicated practical sessions (run online via MS Teams for online students and on-campus for students who selected to join classes at Burwood\Geelong). Please, come prepared so that the class time is used efficiently and fairly for all students in it. Be on time with respect to the specified discussion deadline.
  You will also need to **answer all additional questions** that your tutor may ask you. Questions will cover the lecture notes; so, attending (or watching) the lectures should help you with this **compulsory** discussion part. You should start the discussion as soon as possible as if your answers are wrong, you may have to pass another round, still before the deadline. Use available attempts properly.

Note that we will not accept your solution after **the submission deadline** and will not discuss it after **the discussion deadline**. If you fail one of the deadlines, you fail the task, and this reduces the chance to pass the unit. Unless extended for all students, the deadlines are strict to guarantee smooth and on-time work throughout the unit.

Remember that this is your responsibility to keep track of your progress in the unit that includes checking which tasks have been marked as completed in the OnTrack system by your marking tutor, and which are still to be finalised. When grading your achievements at the end of the unit, we will solely rely on the records of the OnTrack system and feedback provided by your tutor about your overall progress and the quality of your solutions.

Appendix A: Expected Printout

The following provides an example of the output generated from the testing module (Tester.cs) once you have correctly implemented all methods of the Vector<T> class.

```
Test A: Sort integer numbers applying RandomizedQuickSort with the AscendingIntComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
Resulting order: [100,120,122,175,213,236,263,299,312,333,511,596,722,724,752,772,780,958,966,995]
 :: SUCCESS

Test B: Sort integer numbers applying RandomizedQuickSort with the DescendingIntComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
Resulting order: [995,966,958,780,772,752,724,722,596,511,333,312,299,263,236,213,175,122,120,100]
 :: SUCCESS

Test C: Sort integer numbers applying RandomizedQuickSort with the EvenNumberFirstComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
Resulting order: [120,312,780,752,724,966,596,722,100,236,958,772,122,213,263,995,333,299,511,175]
 :: SUCCESS

Test D: Sort integer numbers applying MergeSortTopDown with the AscendingIntComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
```

```
Resulting order: [100,120,122,175,213,236,263,299,312,333,511,596,722,724,752,772,780,958,966,995]
 :: SUCCESS

Test E: Sort integer numbers applying MergeSortTopDown with the DescendingIntComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
Resulting order: [995,966,958,780,772,752,724,722,596,511,333,312,299,263,236,213,175,122,120,100]
 :: SUCCESS

Test F: Sort integer numbers applying MergeSortTopDown with the EvenNumberFirstComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
Resulting order: [772,596,958,752,120,122,724,966,722,100,780,312,236,995,299,175,263,213,511,333]
 :: SUCCESS

Test G: Sort integer numbers applying MergeSortBottomUp with the AscendingIntComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
Resulting order: [100,120,122,175,213,236,263,299,312,333,511,596,722,724,752,772,780,958,966,995]
 :: SUCCESS

Test H: Sort integer numbers applying MergeSortBottomUp with the DescendingIntComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
Resulting order: [995,966,958,780,772,752,724,722,596,511,333,312,299,263,236,213,175,122,120,100]
 :: SUCCESS

Test I: Sort integer numbers applying MergeSortBottomUp with the EvenNumberFirstComparer:
Initial data: [333,236,312,780,100,722,511,966,213,724,122,120,263,175,752,958,596,299,995,772]
Resulting order: [772,596,958,752,120,122,724,966,722,100,780,312,236,995,299,175,263,213,511,333]
 :: SUCCESS


 ------------------- SUMMARY -------------------
Tests passed: ABCDEFGHI
```